Università degli Studi di Palermo

Metodi Numerici Avanzati

Decomposizione a valori singolari (SVD) per il trattamento di immagini mediche

Docenti:
Francomano Elisa
Girfoglio Michele

Autore:
Giuseppe Cucchiara

Corso di Laurea:
LM-32 Ingegneria Informatica

A.A. 2025/2026

Contents

1 Introduzione **3**

2 Decomposizione ai valori singolari **3**
2.1 Cenni Storici 3
2.2 Definizione di SVD 4
2.3 Teorema 1 4
2.4 Algoritmo di Golub-Reinsch 5
2.5 Interpretazione Geometrica dell'SVD 6
2.6 Approssimazione Lower Rank (Teorema di Eckart-Young) 8

3 Compressione delle immagini **8**
3.1 Cosa vuol dire comprimere un immagine? 8
3.2 Rapporto di compressione 13
3.3 Gestione del rumore 15

4 Approssimazione Ottimale tramite Troncamento **16**
4.1 Rimozione del rumore gaussiano 16
4.2 Calcolo del grafico del profilo di intensità 19

5 Complessità Strutturale **22**
5.1 Distinzione tra Immagini Lisce e Rugose 22
5.2 Ricostruzione e Approssimazione di Rango Ridotto 25

6 Costo computazionale **28**
6.1 Analisi del costo computazionale 28

7 Watermarking **28**
7.1 Importanza del Watermarking 28
7.2 Implemetazioni in MatLab 29

8 Metodi con approssimazione a basso rango **34**
8.1 SVD Sottile 35
8.2 SVD Compatta 36
8.3 SVD Troncata 37
8.4 Illustrazione 40

9 Conclusioni **42**

10 Riferimenti **43**

1 Introduzione

L'SVD è un metodo di applicazione matriciale che può avere ampie applicazioni nell'ambito della pratica, in particolare nell'elaborazione per le immagini, poichè consente di scomporre una matrice in componenti principali che possono essere utilizzati in vari scopi. L'SVD gestisce l'elaborazione delle immagini basandosi su due sottospazi, quello dei dati e quello del rumore. Il rumore può essere gestito e filtrato per migliorarne la qualità delle immagini. In questo caso è di fondamentale importanza nell'ambito dell'elaborazione per le immagini mediche poichè l'SVD può essere utilizzata per diversi scopi, come migliorare la qualità delle immagini, ridurre il rumore, estrarre caratteristiche rilevanti, consentendo una visione più chiara delle strutture anatomiche o delle anomalie patologiche. In sostanza, l'SVD permette di rappresentare le informazioni delle immagini in modo più compatto, riducendo la loro dimensione senza compromettere significativamente la qualità.

2 Decomposizione ai valori singolari

2.1 Cenni Storici

In origine, la decomposizione ai valori singolari fu sviluppata da diversi studiosi di geometria differenziale nel corso dei decenni. Eugenio Beltrami nel 1873, e Camille Jordan nel 1874, in modo indipendente tra loro, scoprirono che i valori singolari della forma bilineare, rappresentati in una matrice, formano un insieme completo di invarianti per forme bilineari. Anche James Joseph Sylvester arrivò al risultato della SVD ottenendo risultati simili a quelli ottenuti da Beltrami e Jordan chiamando i valori singolari come moltiplicatori canonici di una matrice. Nel 1907, Erhard Schmidt definì un analogo dei valori singolari per gli operatori integrali, teoria che fu sviluppata successivamente da a Émile Picard nel 1910, che fu il primo a chiamare i numeri *valeurs singulières* e a denotarli σ_k. Nel 1915 il quarto matematico a scoprire la decomposizione a valori singolari fu Léon Autonne che raggiunse la sua formulazione tramite lo studio delle matrici per decomposizione polare. Nel 1936, Carl Eckart e Gale Young presentarono la prima dimostrazione del processo di decomposizione per matrici rettangolari e a valori complessi. I primi metodi pratici per calcolare la SVD risalgono a Ervand Kogbetliantz (1954-1955) e Magnus Hestenes (1958), che usavano approcci simili al metodo di Jacobi basati su rotazioni del piano o rotazioni di Givens. Tuttavia, tali approcci furono rimpiazzati dal metodo di Gene H. Golub e William Kahan utilizzando trasformazioni di Householder.

2.2 Definizione di SVD

Sia $M \in R^{m \times n}$. Allora esistono due matrici ortogonali U e V tali che:

$$M = U \, \Sigma \, V^T$$

dove U è una matrice ortogonale di dimensioni $m \times m$, Σ è una matrice diagonale di dimensioni $m \times n$ e V è anch'essa una matrice ortogonale di dimensioni $n \times n$. Gli elementi della matrice diagonale Σ sono:

$$\Sigma_{ij} = \{ \, 0 \; i \neq j \sigma_i i = j$$

essendo $\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_r \geq 0$.
Gli elementi diagonali della matrice Σ, ordinati in senso decrescente, sono chiamati *valori singolari* di M, e

$$\sigma_i = \sqrt{\lambda_i}$$

dove λ_i sono gli autovalori della matrice

$$B = M^T M.$$

Infine, la decomposizione si può anche scrivere come:

$$M = \sum_{i=1}^{r} \sigma_i \, u_i \, v_i^T$$

dove u_i sono le colonne della matrice U, detti *vettori singolari sinistri*, e corrispondono agli autovettori di MM^T, mentre v_i sono le colonne della matrice V, detti *vettori singolari destri* e corrispondono agli autovettori di $M^T M$.

2.3 Teorema 1

Se risulta che

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > \sigma_{r+1} = \cdots = \sigma_p = 0 \qquad conr < p,$$

dove r è il numero di valori singolari positivi, allora:

- r è il rango di $A \in R^{m \times n}$;
- $$A = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \cdots + \sigma_r u_r v_r^T.$$

Questa formula è anche detta *decomposizione spettrale* di A.
Dunque ogni matrice di rango r si può scrivere come somma di r matrici di

rango 1.
Nel caso di matrici quadrate simmetriche abbiamo visto che

$$A = Q\Lambda Q^T,$$

con Q ortogonale e gli autovalori sulla diagonale di Λ.
La decomposizione

$$A = U\Sigma V^T$$

può essere vista come una sorta di *diagonalizzazione* per matrici arbitrarie, anche non quadrate; si noti tuttavia che per questa *diagonalizzazione* si richiedono due matrici ortogonali, U e V, anziché una sola.

2.4 Algoritmo di Golub-Reinsch

Dal punto di vista teorico, il calcolo della SVD di una matrice $A \in R^{m\times n}$ può essere ricondotto alla determinazione degli autovalori della matrice A^TA. Tuttavia, nelle applicazioni reali e specialmente per matrici di grandi dimensioni, il calcolo diretto tramite il prodotto A^TA è fortemente sconsigliato per due ragioni principali:

1. **Onerosità computazionale:** Il prodotto di matrici A^TA richiede un numero elevato di operazioni, rendendo il processo inefficiente.

2. **Stabilità numerica:** Se la matrice A è malcondizionata, il condizionamento della matrice A^TA aumenta quadraticamente ($cond(A^TA) = [cond(A)]^2$), portando a una perdita significativa di precisione numerica.

Per garantire stabilità e precisione, si utilizza comunemente l'algoritmo di **Golub-Reinsch**. Tale procedura evita il calcolo esplicito di A^TA e si articola nei seguenti passaggi fondamentali:

- **Bidiagonalizzazione:** Si trasforma la matrice A in una forma bidiagonale superiore B mediante una successione di trasformazioni ortogonali di Householder:

$$B = P^{(n)}\left(\dots\left(P^{(2)}\left(P^{(1)}AH^{(1)}\right)H^{(2)}\right)\dots\right)H^{(n-2)} \tag{1}$$

  dove $P^{(k)}$ e $H^{(s)}$ sono matrici di Householder.

- **Diagonalizzazione:** Si procede a diagonalizzare la matrice tridiagonale simmetrica B^TB applicando il metodo QR per estrarre i valori singolari di A. Sfruttando la relazione:

$$A^TA = H(B^TB)H^TA^TA = (HQ)\Lambda(Q^TH^T) \tag{2}$$

  è possibile isolare la matrice diagonale Λ.

- **Calcolo dei vettori singolari:** Si definisce la matrice dei vettori singolari destri come $V = HQ$ e si completa il calcolo determinando i vettori singolari sinistri u_i.

2.5 Interpretazione Geometrica dell'SVD

Consideriamo una matrice quadrata $M \in R^{m \times m}$. Le matrici U e $V^\top$ possono essere scelte anch'esse quadrate $m \times m$. Le tre matrici della decomposizione singolare, U, Σ e $V^\top$, possono essere interpretate come una sequenza di trasformazioni che, insieme, definiscono una trasformazione lineare

$$x \mapsto Mx,$$

sullo spazio vettoriale R^m.
Le matrici U e $V^\top$ rappresentano rotazioni o riflessioni dello spazio, mentre Σ rappresenta il ridimensionamento di ciascuna coordinata x in base ai fattori σ_i. Quindi, la decomposizione SVD scompone qualsiasi trasformazione lineare dello spazio R^m in una composizione di tre trasformazioni geometriche: una rotazione o riflessione ($V^\top$), seguita da un ridimensionamento (Σ), seguita da un'altra rotazione o riflessione (U). Si puo dimostrare che se il determinante della matrice M è positivo, allora le si possono scegliere le matrici U e $V^\top$ in modo tale che entrambe effettuino rotazioni sia con riflessioni che senza. Se il determinate di M fosse negativo, otterremmo una rotazione con riflessione e una senza. Se il determinate fosse uguale a 0, avremmo potuto scegliere in maniera indipendente il tipo di rotazione. Questo concetto viene generlizzato nello spazio euclideo n-dimensionale, i cui valori singolari possono essere interpretati come la grandezza dei semiassi di un ellisse. In Figura 1 riportiamo le seguenti trasformazioni dell'SVD.

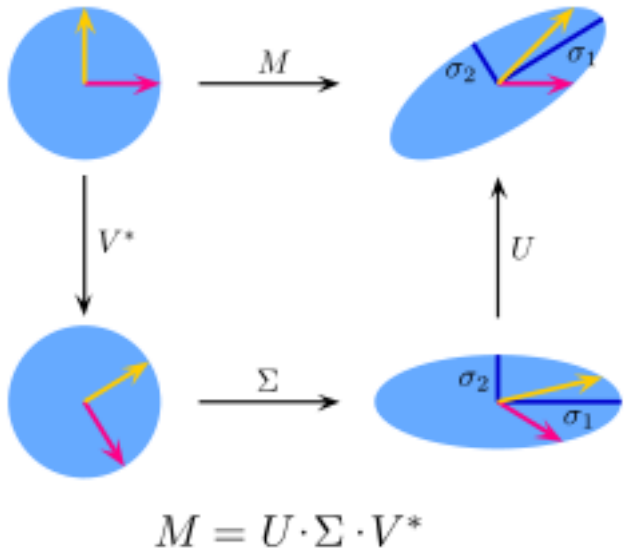

Figure 1: Rotazioni

Vediamo che viene effettuata una rotazione $V^\top$, al secondo passo uno scaling di Σ lungo gli assi delle coordinate e la rotazione finale in U. Presa invece una matrice $M \in R^{m\times n}$ con $m \neq n$, la trasformazione sarebbe stata lineare da R^m a R^n. Quindi le matrici U e $V^\top$ possono essere scelte come rotazioni o riflessioni in R^m a R^n.

2.6 Approssimazione Lower Rank (Teorema di Eckart-Young)

Sia $A = U\Sigma V^T$ con $A \in R^{m\times n}$ e sia r il rango di A. Fissato $k < r$, si definisce

$$A_k = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \cdots + \sigma_k u_k v_k^T.$$

La matrice A_k è un'approssimazione di A di rango k.
L'errore che si commette approssimando A con A_k è

$$\min_{B\in\mathcal{B}} \|A - B\|_2 = \|A - A_k\|_2 = \sigma_{k+1},$$

essendo

$$\mathcal{B} = \{B \in R^{m\times n} \mid rank(B) = k\}.$$

La possibilità di approssimare, in modo ottimale, una matrice arbitraria con una di rango "basso" permette la compressione di grandi insiemi di dati, tipicamente affetti da errore (o "rumore"). Questo teorema è di fondamentale importanza per la compressione e elaborazione delle immagini.

3 Compressione delle immagini

3.1 Cosa vuol dire comprimere un immagine?

Comprimere una immagine significa riscostruire un'immagine con un numero ridotto di informazioni. Per memorizzare un'immagine su un supporto digitale, la si deve scomporre in piccoli quadratini e associare ad ogni quadratino un certo numero di informazioni utili a ricostruire la parte di figura contenuta nel quadratino stesso. Queste informazioni sono valori numerici in genere ottenuti come media di informazioni relative a tutto il quadratino e sono memorizzate in un cosiddetto *pixel*.
Vediamo una prima implementazione in Matlab:

```
% Caricamento del file DICOM
file = '000007.dcm';

% Lettura dell'immagine
image = dicomread(file);

% Converti l'immagine al tipo double per l'elaborazione con SVD
[U, S, V] = svd(double(image));

% Definizione dei Livelli di Compressione
valori_singolari = [10, 25, 50, 75, 100];
```

```

% Valori singolari
sigma = diag(S);

% Energia totale
energia_totale = sum(sigma.^2);

% Preallocazione
informazione_catturata = zeros(size(valori_singolari));

% Calcolo percentuale di informazione
for i = 1:length(valori_singolari)
    K = valori_singolari(i);
    informazione_catturata(i) = ...
        sum(sigma(1:K).^2) / energia_totale * 100;
end

% Visualizzazione dei Risultati
figure;
set(gcf, 'Name', 'Compressione Immagine TC con SVD');

% Aggiungiamo un subplot per l'immagine originale
subplot(2, 3, 1);
% Visualizziamo l'immagine originale
imshow(image, []);
title('1. Immagine Originale');
colormap gray;

% Ciclo per ricostruire e visualizzare le immagini compresse
for i = 1:length(valori_singolari)
    K = valori_singolari(i);

    % Riduciamo la dimensione delle matrici per l'approssimazione a basso rango
    U_k = U(:, 1:K);
    S_k = S(1:K, 1:K);
    V_k = V(:, 1:K);

    % Ricostruiamo l'immagine utilizzando la formula dell'approssimazione
    % La matrice risultante e' di tipo double.
    immagine_ricostruita = U_k * S_k * V_k';

    % Aggiungiamo un subplot per l'immagine ricostruita
    subplot(2, 3, i + 1);

    % Visualizziamo l'immagine ricostruita
    imshow(immagine_ricostruita, []);
    title(['K = ', num2str(K)]);
    colormap gray;
end
```

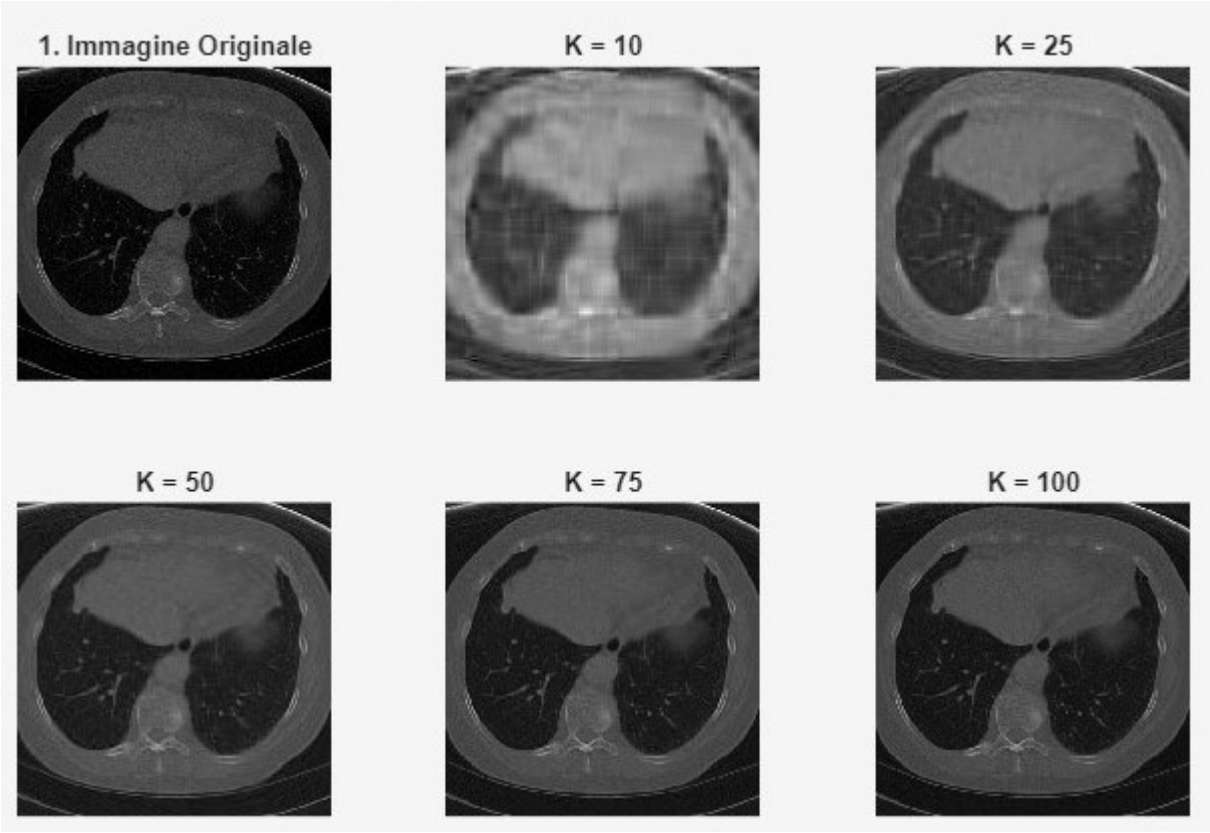

Figure 2: Risultati compressioni Immagini

Abbiamo appena implementato un SVD a un immagine che rappresenta una TAC in cui vediamo che al variare dell'indice k, otteniamo un immagine più nitida e più completa che ne garantisce una diagnosi corretta. Nel nostro caso, abbiamo un immagine più chiara di una Neoplasia Maligna al Lobo Inferiore Destro (Figure 2). Con K = 100, la diagnosi di neoplasia al lobo inferiore 'e accurata quanto l'originale, ma con meno dati.

```
figure;
hold on;
plot(valori_singolari, informazione_catturata, '-o', 'LineWidth', 1.5);
xlabel('Numero di Valori Singolari Mantenuti');
ylabel('Informazione catturata (%)');
title('Informazione catturata vs Numero di Valori Singolari Mantenuti');
ylim([90 100]);
hold off;

figure;
hold on;
bar(valori_singolari, informazione_catturata);
xlabel('Numero di Valori Singolari Mantenuti');
ylabel('Informazione catturata (%)');
title('Informazione catturata vs Numero di Valori Singolari Mantenuti');
ylim([0 100]);
hold off;
```

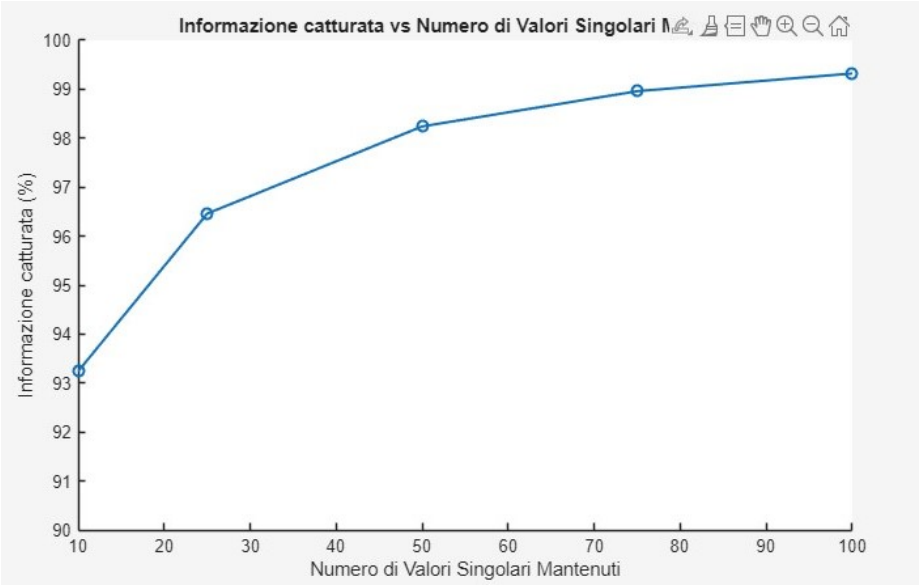

Figure 3: Grafico Informazione Catturata

I grafici mostrati in Figures 3 e 4 viene mostrata la variazione dell'infomazione catturata al variare dei valori singolari. Questi grafici confermano l'efficienza dell'SVD nel trattenere energia del segnale. Già con pochi valori singolari si supera il 93% dell'informazione totale.

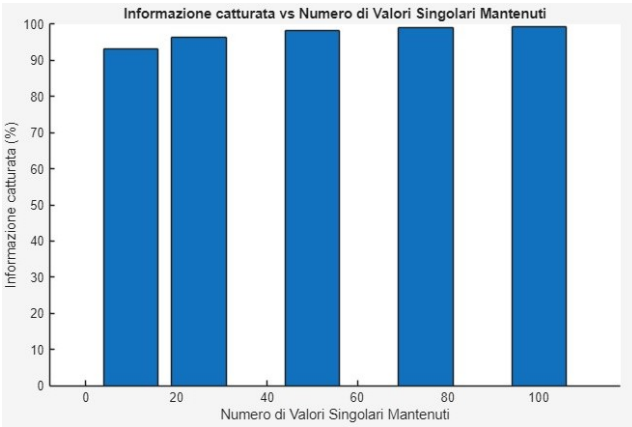

Figure 4: Grafico Informazione Catturata 2

3.2 Rapporto di compressione

È possibile definire una *Compression Ratio* che esprime il grado di compressione ottenuto applicando la SVD data da:

$$R = \frac{k * n + k + k * m}{m * n} * 100$$

Con R che rappresenta l'indice di compressione espresso in percentuale, k rappresenta il numero di valori singolari mantenuti, m e n sono le dimensioni originali dell'immagine. Vediamo un implementazione con il rapporto di compressione:

```
file = '000007.dcm';
image = dicomread(file);
% SVD
[U, S, V] = svd(double(image));
m = size(image, 1);
n = size(image, 2);

valori_singolari_da_visualizzare = [10, 20, 50, 100];

% Inizializzazione corretta
rapporto_compressione = zeros(1,length(valori_singolari_da_visualizzare));
rango_di_approssimazione = zeros(1,length(valori_singolari_da_visualizzare));

figure;
for i = 1:length(valori_singolari_da_visualizzare)
k = valori_singolari_da_visualizzare(i);
immagine_compressa = U(:, 1:k)*S(1:k, 1:k)* V(:,1:k)';

% Calcolo del rapporto di compressione (metrica di memoria occupata)
compression_ratio = (((n * k) + k + (m * k))/(m * n))*100;
rapporto_compressione(i) = compression_ratio;
```

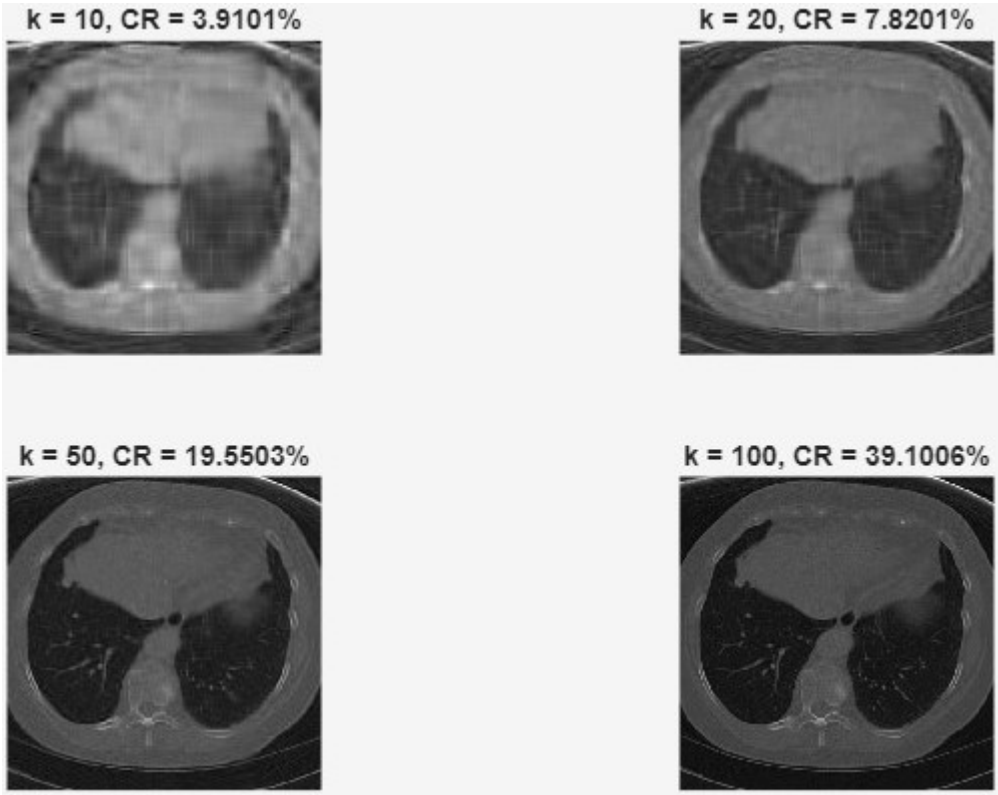

Figure 5: Risultati ottenuti con il troncamento

```
rango_di_approssimazione(i) = k;

subplot(2, 2, i);
imshow(immagine_compressa, []);
title(['k = ' num2str(k) ', CR = ' num2str(compression_ratio) '%']);
end
```

Il risultato finale ottenuto in Figure 5 è la visualizzazione di quattro immagini approssimate in una singola figura. Ogni immagine ha un diverso grado di fedeltà e un diverso Rapporto di Compressione: all'aumentare di k (da 10 a 100), la qualità visiva dell'immagine ricostruita migliora, ma il rapporto di compressione peggiora, infatti notiamo come a k = 100 il CR sia circa il 39%, garantendo un'ottima fedeltà diagnostica pur riducendo il peso del file di oltre il 60%. Questa operazione di riduzione dei dati tramite la selezione dei componenti principali è definita matematicamente *Troncamento*.

3.3 Gestione del rumore

Un ruolo molto importante nell'elaborazione delle immagini è la gestione del *rumore*. Cos'è il rumore???? Il rumore è un'alterazione casuale dei pixel che fa apparire l'immagine granulosa, poco nitida e con puntini colorati o grigi. In poche parole, è un disturbo visivo dell'immagine compromettendo diverse caratteristiche dell'immagine. Per ridurre il rumore è necessario filtrarlo dividendo il nostro spazio vettoriale in due sottospazi distinti in modo tale da migliorare la manipolazione e l'analisi dell'informazione presenti nelle immagini. Il tutto lo dimostriamo tramite un implementazione in MatLab:

```
file = '000007.dcm'; % File DICOM
image = dicomread(file);
% Conversione in double
[U, S, V] = svd(double(image));
% Numero di componenti per il sottospazio dei dati dell'immagine
K = 20;
% Calcoliamo l'approssimazione a rango K
sottospazio_dati_immagine = U(:, 1:K) * S(1:K, 1:K) * V(:, 1:K)';
% Calcolo del Rumore Estratto
rumore_estratto = double(image) - double(sottospazio_dati_immagine);
figure('Name', 'Riduzione del rumore con SVD');
% Visualizzazione dell'Immagine Originale
subplot(1, 3, 1);
imshow(image, []);
title('Immagine Originale');

% Immagine Filtrata
subplot(1, 3, 2);
imshow(sottospazio_dati_immagine, []);
title(['Immagine Filtrata (K = ' num2str(K) ')']);

% Rumore Estratto
subplot(1, 3, 3);
imshow(rumore_estratto, []);
title('Rumore Estratto');
```

Da questa implementazione possiamo notare in Figure 6 che l'immagine originale viene scomposta, le 20 componenti più importanti mantengono l'informazione utile, creando l'immagine filtrata (pulita), e la parte rimanente scartata diventa il rumore estratto.

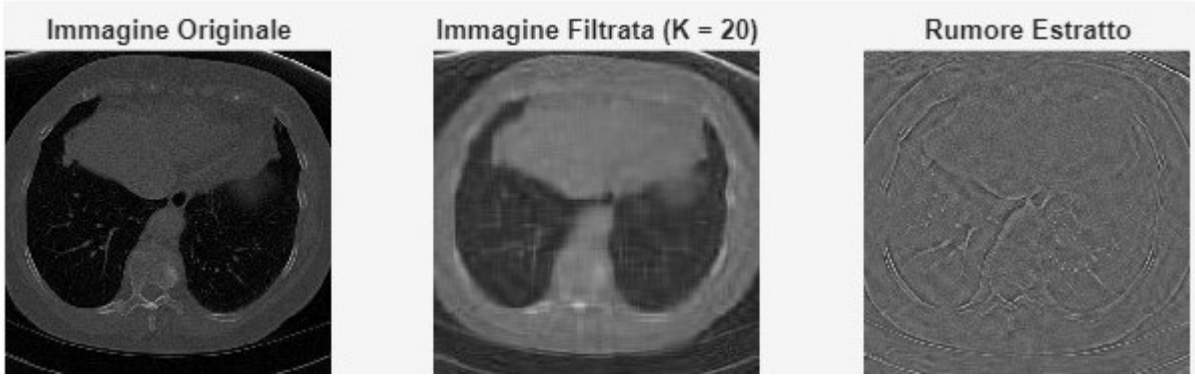

Figure 6: Risultati ottenuti con l'estrazione del rumore

4 Approssimazione Ottimale tramite Troncamento

4.1 Rimozione del rumore gaussiano

Il cuore dell'efficacia dell'SVD nel trattamento di matrici ad alto rango, risiede nel concetto di troncamento a rango k. Un'immagine è trattata come una matrice A, che la SVD scompone nei suoi componenti principali. Il troncamento consiste nel selezionare solo i k valori singolari maggiori e i relativi vettori singolari, ottenendo la migliore approssimazione di rango k, $\hat{A}_k$, sia in norma spettrale che in norma di Frobenius. Nel contesto delle applicazioni reali, non si agisce direttamente sulla matrice ideale A, ma su una stima grezza $\hat{A}$, che è spesso una versione di A perturbata dal rumore:

$$\hat{A} = A + E$$

dove E è la matrice di errore o rumore. Un caso particolarmente rilevante è quando l'errore è modellato dal rumore gaussiano. Il rumore gaussiano è un modello di disturbo stocastico fondamentale nel processamento delle immagini, spesso utilizzato per simulare il rumore termico o i problemi di illuminazione.

1. **Distribuzione e Caratteristiche**: Questo tipo di rumore è caratterizzato da una distribuzione simmetrica (distribuzione normale o gaussiana) e assume valori casuali con una media di zero. La sua deviazione standard (σ) ne determina l'intensità (la "larghezza" della distribuzione), e per questo è talvolta chiamato rumore additivo bianco gaussiano (AWGN).

2. **Natura Casuale**: Ogni pixel nell'immagine $\hat{A}$ è influenzato da un valore casuale, indipendente dagli altri, estratto da questa distribuzione, aggiungendo una quantità di rumore.

3. **Effetto sulle Immagini**: Il rumore gaussiano tende a produrre una sorta di "nebbia" o "grana" sottile sull'immagine, rendendo meno definiti i confini tra i dettagli e introducendo un aspetto leggermente sfocato.

4. **Test e Mitigazione**: È spesso usato come modello di base per testare algoritmi di riduzione del rumore. Filtri classici (come il filtro gaussiano o il filtro passa-basso) o tecniche basate sulla media dei pixel circostanti possono essere usati per mitigarne l'impatto.

Quando l'immagine osservata $\hat{A}$ è contaminata da rumore gaussiano, l'efficacia del troncamento SVD risiede proprio nel gestire il compromesso ottimale tra due fonti di errore:

1. **Bias (Errore di Approssimazione)**: L'errore irriducibile $\|A - A_k\|_F$ introdotto dal troncamento forzato della matrice originale A a un rango inferiore k.

2. **Varianza (Errore di Rumore)**: Il contributo del rumore E che viene amplificato o mantenuto nel processo.

L'applicazione dell'SVD troncata su $\hat{A}$ produce un'approssimazione finale $\hat{A}_k$ che si avvicina ad A con un errore relativo moltiplicativo in norma di Frobenius, a patto che l'errore iniziale sia contenuto. Per il denoising di matrici ad alto rango, il metodo consente di recuperare il segnale A dalle osservazioni disturbate senza assumere che A sia esattamente a basso rango. L'intuizione chiave è che, se i valori singolari di A (che rappresentano l'importanza dei componenti del segnale) decrescono rapidamente, i valori singolari più piccoli di $\hat{A}$ sono dominati dal rumore E.

Se si sceglie un rango k tale che la norma del rumore ($\|E\|_2 \approx \nu$) sia significativamente inferiore al $(k+1)$-esimo valore singolare di A (ovvero $\nu < \sigma_{k+1}(A)$), il troncamento non solo filtra il rumore, ma assicura che l'errore finale sia essenzialmente controllato dalla parte del segnale A che è stata volutamente scartata, $\|A - A_k\|_F$, a meno di un fattore moltiplicativo $(1 + O(\epsilon))$.

Questo approccio, che richiede solo una singola SVD troncata (computazionalmente efficiente), rappresenta una strategia robusta per recuperare il segnale essenziale da osservazioni contaminate o incomplete, anche quando la matrice originale è ad alto rango. Implementiamo in MatLab:

```
% Caricamento e Preparazione dell'Immagine Originale
file = 'COVID-101.png';

% Carica l'immagine medica e converti in double
image=imread(file)

% Carica l'immagine medica e converti in double
A = double(image);
[m, n] = size(A);

% Aggiunta del Rumore Gaussiano
livello_rumore = 30; % Fattore di scala per il rumore
rumore = livello_rumore * randn(m, n);

```

```
A_rumorosa = A + rumore;

% Definizione del Rango di Troncamento
T = 50;

% SVD e Troncamento sulla Matrice Rumorosa
[U_hat, S_hat, V_hat] = svd(A_rumorosa);

% Troncamento e ricostruzione con solo i primi T componenti
U_T = U_hat(:, 1:T);
S_T = S_hat(1:T, 1:T);
V_T = V_hat(:, 1:T);

img_recuperata = U_T * S_T * V_T';

% Visualizzazione
figure;

subplot(1, 3, 1);
imshow(A, []);
title('Immagine Originale');

subplot(1, 3, 2);
imshow(A_rumorosa, []);
title('Immagine Rumorosa');

subplot(1, 3, 3);
imshow(img_recuperata, []);
title(['Immagine Recuperata (T=', num2str(T), ')']);
```

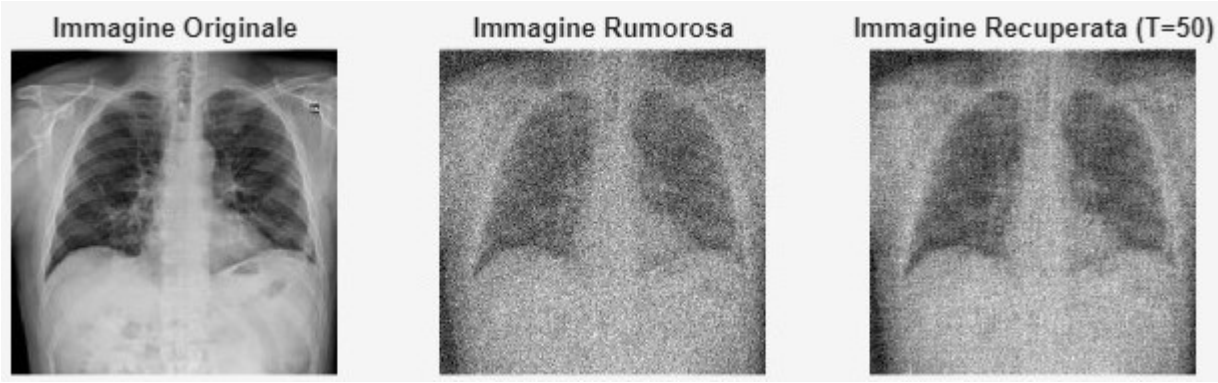

Figure 7: Troncamento con rumore gaussiano

Il risultato finale ottenuto in Figure 7 ci mostra che l'immagine originale è molto vicina all'immagine recuperata però ci sono differenze notevoli tra l'immagine recuperata e quella rumorosa. Questo perchè l'SVD troncata ha agito come filtro, rimuovendo la maggior parte del rumore.

4.2 Calcolo del grafico del profilo di intensità

Confrontiamo adesso le tre immagini in Figure 7 attraverso un grafico del profilo di intensità.

```
% Selezione della 100esima riga
riga = 100;

% Profili di intensita'
profilo_originale  = A(riga, :);
profilo_rumoroso   = A_rumorosa(riga, :);
profilo_recuperato = img_Recuperata(riga, :);

% Grafico del profilo di intensita'
figure;
plot(profilo_originale, 'b', 'LineWidth', 1.5);
hold on;
plot(profilo_rumoroso, 'r', 'LineWidth', 1);
plot(profilo_recuperato, 'g', 'LineWidth', 1.5);
hold off;

grid on;
xlabel('Indice del pixel');
ylabel('Intensit ');
title("Confronto del Profilo di Intensit  lungo una Riga dell'immagine");
legend('Originale', 'Rumorosa', 'Recuperata');
```

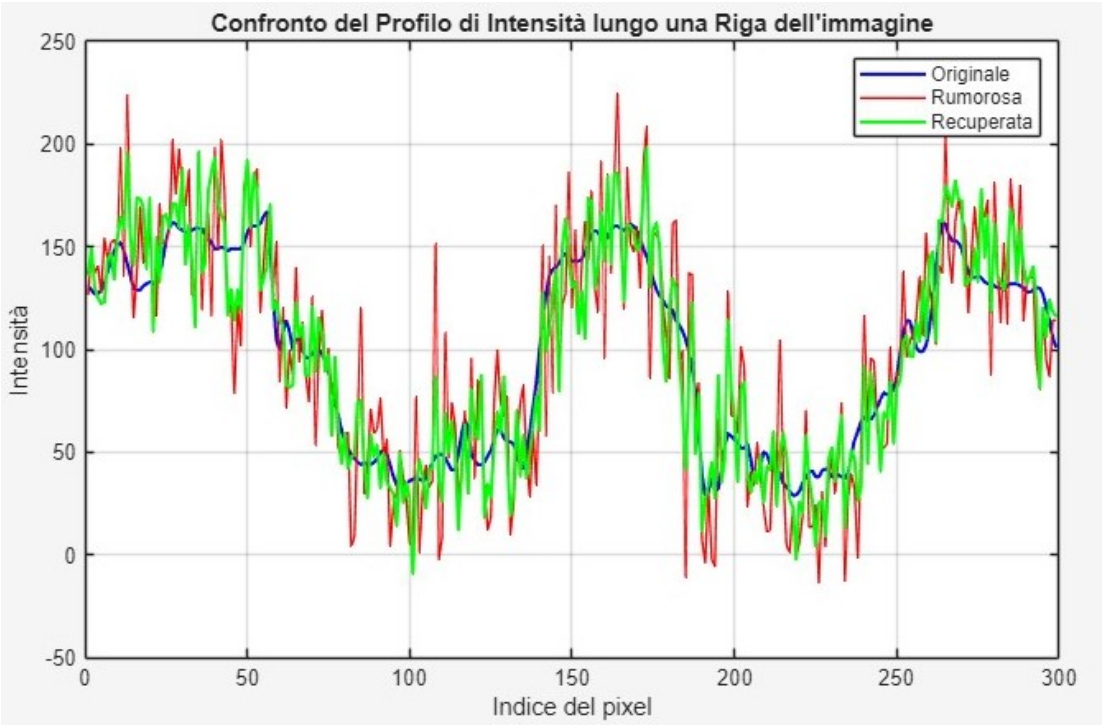

Figure 8: Grafico del profilo intensità

Il profilo di intensità calcolato sulla 100ª riga mostrato in Figure 8 evidenzia come il rumore gaussiano introduca forti variazioni locali, mentre la ricostruzione tramite SVD troncata consente di ottenere un profilo più regolare e vicino a quello dell'immagine originale.

5 Complessità Strutturale

Un fattore determinante per l'efficacia della compressione e del filtraggio tramite SVD è la complessità intrinseca della matrice immagine. L'andamento dei valori singolari riflette direttamente tale complessità: in genere, si osserva una predominanza dei primi valori singolari (quelli di entità maggiore), che racchiudono la struttura portante dell'immagine, seguita da un decadimento verso i valori minori, che spesso rappresentano dettagli fini o rumore. Tale andamento non è uniforme, ma dipende strettamente dalle peculiarità specifiche del contenuto visivo e dalle statistiche del rumore presente.

5.1 Distinzione tra Immagini Lisce e Rugose

Come osservato sperimentalmente, nelle **immagini lisce** (prive di rumore), i valori singolari tendono a presentare una **rapida diminuzione** (decadimento esponenziale). Questo accade perché l'informazione strutturale è altamente correlata e può essere sintetizzata da poche componenti principali. In termini pratici, ciò significa che:

- **Elevata Comprimibilità:** è possibile ricostruire l'immagine con un numero molto esiguo di valori singolari (k basso) senza introdurre artefatti visibili.
- **Concentrazione Energetica:** la quasi totalità dell'energia del segnale è contenuta nei primi termini della decomposizione.

Al contrario, nelle **immagini rugose**, i valori singolari decrescono molto più lentamente. Il rumore, infatti, introduce componenti ad alta frequenza che si distribuiscono lungo tutto lo spettro dei valori singolari, creando una "coda" di valori non trascurabili. In questo caso si osserva:

- **Complessità di Troncamento:** un valore di k troppo basso comporterebbe una perdita drastica di dettagli, poiché la SVD non riuscirebbe a distinguere tra la "rugosità" utile (dettaglio clinico) e quella del rumpre.
- **Distribuzione dell'Informazione:** l'energia è dispersa su un numero maggiore di componenti, rendendo l'immagine meno adatta a una compressione spinta.

```
% Caricamento dell'Immagine Originale
file = 'COVID-101.png';
% Caricamento e conversione immediata in double
image = double(imread(file));
% Creazione delle due varianti per l'analisi
% Variante "Liscia": applicazione di filtro Gaussiano
% (rimozione alte frequenze)
immagine_liscia = imgaussfilt(image, 2);
% Variante "Rugosa": aggiunta di rumore gaussiano
```

```
% (introduzione alte frequenze)
immagine_rugosa = imnoise(uint8(image), 'gaussian', 0, 0.05);
immagine_rugosa = double(immagine_rugosa);
% Calcolo SVD per le due varianti
[U_l, S_l, V_l] = svd(immagine_liscia);
[U_r, S_r, V_r] = svd(immagine_rugosa);
% Estrazione valori singolari per il confronto (vettori diagonali)
s_liscia = diag(S_l);
s_rugosa = diag(S_r);
% Confronto Visivo tra le varianti analizzate
figure('Name', 'Confronto Varianti Immagine');
subplot(1,2,1);
imshow(uint8(immagine_liscia));
title('Immagine Liscia');
subplot(1,2,2);
imshow(uint8(immagine_rugosa));
title('Immagine Rugosa');
% Decadimento dei Valori Singolari
figure('Name', 'Analisi Valori Singolari');
semilogy(s_liscia(1:100), 'b', 'LineWidth', 2); hold on;
semilogy(s_rugosa(1:100), 'r', 'LineWidth', 2);
title('Analisi Strutturale: Decadimento Valori Singolari');
xlabel('Indice k');
ylabel('Valore Singolare \sigma_k');
legend('Immagine Liscia', 'Immagine Rugosa');
grid on;
```

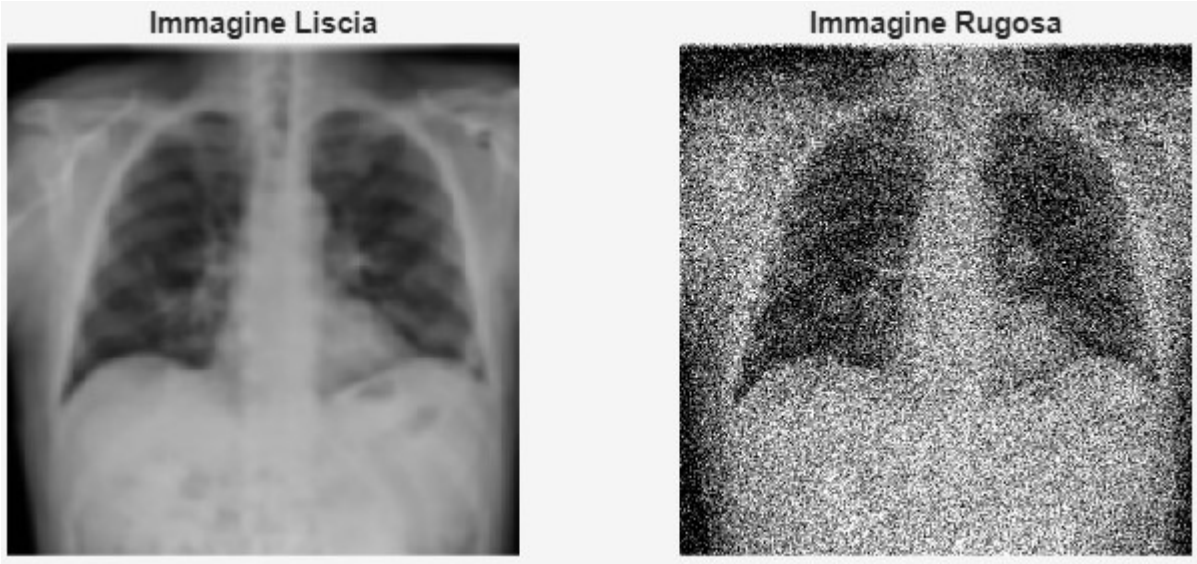

Figure 9: Differenza tra immagine liscia e rugosa

In Figure 10 viene mostrato il decadimento dei valori singolari di un'immagine priva di rumore rispetto ad un'immagine rumorosa:

- **Immagine Liscia:** I valori singolari σ_k decrescono in modo estremamente rapido verso lo zero. Tale andamento indica una **bassa complessità informativa** e un'elevata correlazione spaziale tra i pixel. Matematicamente, l'immagine può essere considerata a *rango numerico ridotto*: l'informazione strutturale dominante, come il parenchima polmonare e le strutture costali, è concentrata in un sottospazio di dimensione $k \ll n$. Di conseguenza, una ricostruzione di rango inferiore è sufficiente a preservare l'integrità anatomica del paziente.

- **Immagine Rugosa:** La curva relativa alla variante rumorosa si mantiene costantemente al di sopra di quella liscia, specialmente per indici k elevati. Questo scarto rappresenta la **"spesa energetica"** indotta dal rumore gaussiano: trattandosi di una componente stocastica incoerente, il rumore non può essere compresso efficacemente dalla SVD. L'algoritmo è dunque costretto a distribuire questa informazione "disordinata" su un numero elevato di componenti, generando una *coda* dello spettro più spessa e piatta, che ostacola il processo di approssimazione ottima.

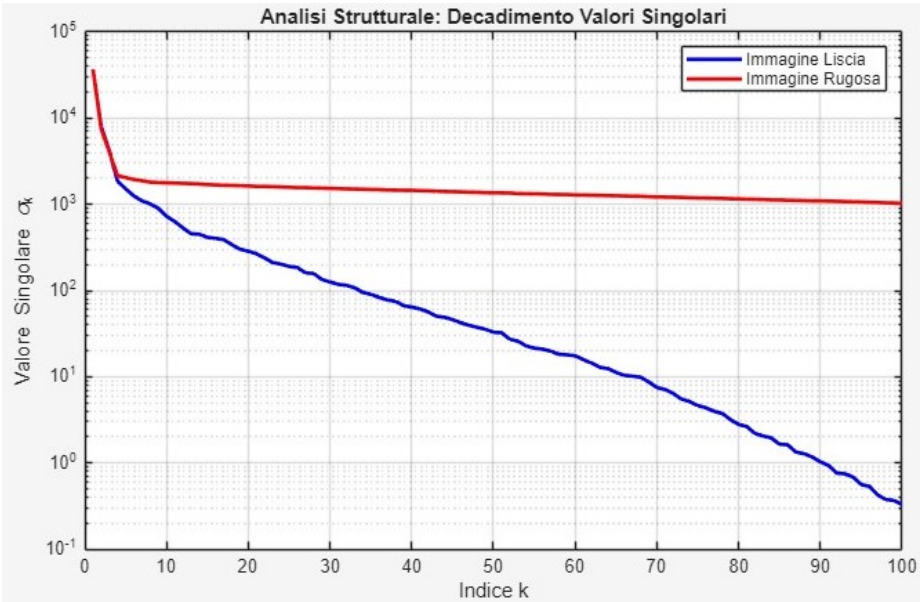

Figure 10: Decadimento Valori Singolari

5.2 Ricostruzione e Approssimazione di Rango Ridotto

Dopo aver analizzato il comportamento dello spettro dei valori singolari, è necessario valutare l'impatto pratico della struttura dell'immagine sulla sua ricostruzione. In questa fase, applichiamo approssimazione low-rank per ottenere un'approssimazione di rango k delle due varianti (liscia e rugosa).
L'obiettivo è osservare come la medesima soglia di troncamento, fissata in questo esperimento a $k = 50$, influenzi la qualità dell'immagine in base alla sua complessità strutturale originale. Questo test permette di verificare empiricamente la capacità della SVD di isolare l'informazione anatomica coerente dalle componenti di disturbo.

```
% Scegliamo un rango ridotto
k = 50;
% Ricostruzione dell'immagine liscia
img_liscia_rec = U_l(:,1:k) * S_l(1:k,1:k) * V_l(:,1:k)';

% Ricostruzione dell'immagine rugosa
img_rugosa_rec = U_r(:,1:k) * S_r(1:k,1:k) * V_r(:,1:k)';

% Visualizzazione del confronto delle ricostruzioni
figure('Name', ['Ricostruzione con k = ', num2str(k)]);
subplot(1,2,1);
imshow(uint8(img_liscia_rec));
title(['Liscia Ricostruita (k=', num2str(k), ')']);
```

```

subplot(1,2,2);
imshow(uint8(img_rugosa_rec));
title(['Rugosa Ricostruita (k=', num2str(k), ')']);
```

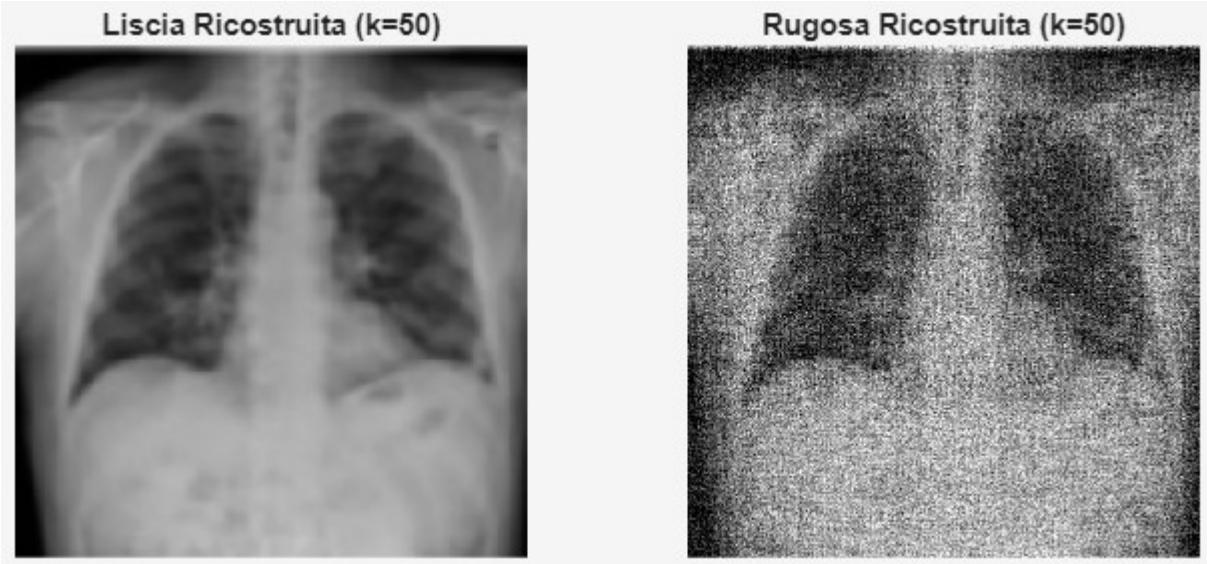

Figure 11: Ricostruzione immagine liscia e rugosa

La ricostruzione di rango ridotto con $k = 50$ evidenzia la differente capacità di sintesi della SVD sulle due varianti. Mentre l'immagine liscia mantiene un'elevata fedeltà strutturale, confermando la concentrazione del segnale nei primi valori singolari, la variante rugosa subisce una perdita di definizione più marcata. Tuttavia, il troncamento opera un filtraggio intrinseco, eliminando le componenti stocastiche ad alta frequenza localizzate nella coda dello spettro.

6 Costo computazionale

L'SVD effettua tantissime operazioni specie se vengono utilizzati dataset di grandissime dimensioni, il che ci dice che il costo computazionale è elevato. Il che ci pone dei limiti nel calcolo della SVD.

6.1 Analisi del costo computazionale

Per calcolare il costo computazionale della SVD, partiamo prima dalle dimensioni delle matrici. Nel caso generale prendiamo $m \times n$ e il numero dei valori singolari K che vogliamo analizzare. Ora partiamo dal calcolo delle matrici coinvolte U, S e V con un costo $O(mn^2)$; adesso calcoliamo il costo computazionale della riduzione delle immagini. In questo caso il costo è di $O(mk + k^2 + nk)$ perchè manteniamo solo i primi k valori singolari modificando solo i vettori colonna della matrice U, le prime $k \times k$ diagonali di S e le prime K colonne di V. Il costo di ricostruzione delle immaagini è lo stesso di quello della riduzione poichè vengono effettuate moltiplicazioni sulle matrici U, S e V^T. Il costo totale è di $O(mn^2 + mk + k^2 + nk)$, il che ci dimostra che è molto costosa come implementazione.

7 Watermarking

7.1 Importanza del Watermarking

L'implementazione di tecniche di watermarking nelle immagini mediche risponde alla cruciale necessità di garantire l'autenticità, l'integrità e la legittima proprietà dei dati clinici, specialmente in contesti di telemedicina e archiviazione elettronica. Quindi nel caso della SVD si adatta molto bene per le sue proprieta':

- **Massima Robustezza e Stabilità**: I valori singolari (S) di una matrice riflettono le proprietà algebriche fondamentali dell'immagine, rappresentandone l'energia intrinseca. La modifica di questi valori per l'incorporazione del *watermark* garantisce che l'informazione sia distribuita uniformemente su tutta la matrice. Questa distribuzione rende il *watermark* eccezionalmente **robusto** contro attacchi comuni e manipolazioni, un requisito vitale per i registri medici che devono resistere a compressioni o modifiche.

- **Alta Impercettibilità (Invisibilità)**: La SVD concentra la maggior parte dell'energia dell'immagine nei primi valori singolari. L'incorporazione di un *watermark* con un fattore di scala (α) ridotto assicura che il cambiamento visivo nell'immagine sia minimo. L'alto PSNR (*Peak Signal-to-Noise Ratio*) raggiunto da questi schemi conferma che il *watermark* è **invisibile**, preservando l'integrità visiva per i medici e prevenendo la compromissione dell'accuratezza diagnostica.

- **Adattabilità all'Elaborazione Dati**: A differenza di altre trasformate con basi fisse, la SVD utilizza basi ortogonali non fisse, fornendo una rap-

presentazione ottimale per qualsiasi blocco di dati. Il principio generale della SVD garantisce che, anche in caso di *watermarking* globale, i cambiamenti mantengano un'ottima correlazione con la struttura geometrica e l'energia dell'immagine medica.

7.2 Implemetazioni in MatLab

Implementiamo in MatLab:

```
% Caricamento Immagine DICOM
file = '000007.dcm';
image = dicomread(file);
img = double(image(:,:,1)); % Slice 2D

w_data = dicomread(file);
w = double(w_data(:,:,1));

% 2. Parametri e SVD
alpha = 0.005;
[U, S, V] = svd(img);
w_r = imresize(w, size(S));

% Otteniamo Uw e Vw (la "forma") e Sw (l'energia) del watermark
[Uw, Sw, Vw] = svd(w_r);

% Inserimento coerente:
S_wt = S + alpha * Sw;
w_img = U * S_wt * V';

% Visualizzazione Risultati Inserimento
w_img_u8 = uint8(255 * (w_img - min(w_img(:))) / (max(w_img(:)) - min(w_img(:))));
img_u8 = uint8(255 * (img - min(img(:))) / (max(img(:)) - min(img(:))));
w_u8 = uint8(255 * (w - min(w(:))) / (max(w(:)) - min(w(:))));

figure(1);
subplot(1,3,1); imshow(img_u8); title('Immagine Originale');
subplot(1,3,2); imshow(w_img_u8); title('Immagine con Watermark');
subplot(1,3,3); imshow(w_u8); title('Watermark');
% continua.....
```

L'analisi visiva della coppia `Immagine Originale` e `Immagine con Watermark` in Figure 12 dimostra l'eccellente invisibilità (o impercettibilità) del watermark. L'occhio umano, e presumibilmente l'occhio clinico, non riesce a distinguere l'immagine ospite modificata dall'originale. Questo risultato è direttamente attribuibile al piccolo fattore di scala (α) scelto, che assicura che il contributo energetico del watermark ai valori singolari dell'immagine medica rimanga trascurabile, salvaguardando l'integrità diagnostica.

Figure 12: Prima figura col Watermark

Di seguito viene riportata in Figure 13 la struttura grafica di questa estrazione:

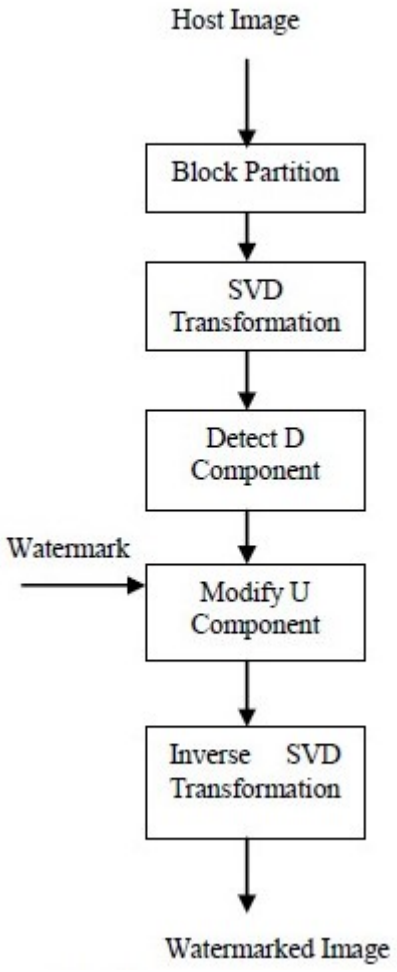

Figure 13: Illustrazione Grafica

```
%.....continuazione
[~, S_w_est, ~] = svd(w_img); % SVD dell'immagine marchiata
Sw_recuperata = (S_w_est - S) / alpha; % Isolo i valori singolari

w_estratto_finale = Uw * Sw_recuperata * Vw';

% Visualizzazione Risultati Estrazione
norm_w_e = uint8(255 * (w_estratto_finale - min(w_estratto_finale(:))) ...
    / (max(w_estratto_finale(:)) - min(w_estratto_finale(:))));

figure(2);
subplot(1,2,1); imshow(w_u8); title('Watermark Originale');
subplot(1,2,2); imshow(norm_w_e); title('Watermark Estratto');
```

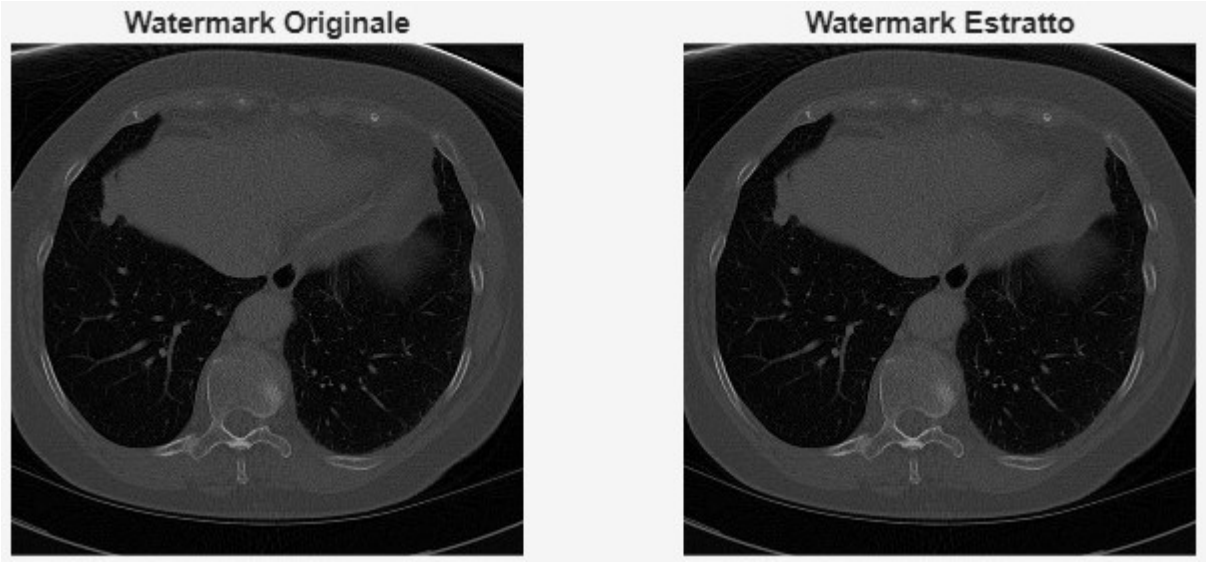

Figure 14: Confronto tra Watermark originale e Watermark Estratto

L'esame della coppia `Watermark Originale` e `Watermark Estratto` in Figure 14 conferma la fedeltà e la robustezza del meccanismo. Il watermark estratto si presenta come una replica quasi perfetta dell'originale, con solo minime imprecisioni dovute alla natura numerica dell'elaborazione. Questo successo indica che l'SVD ha efficacemente distribuito l'informazione del watermark all'interno dei valori singolari, proteggendola da qualsiasi perdita significativa.

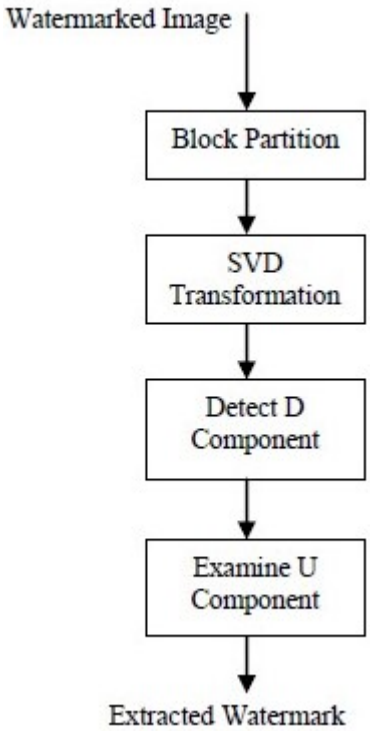

Figure 15: Illustrzione Grafica Watermark Estratto

8 Metodi con approssimazione a basso rango

Essendo l'SVD un'implementazione molto costosa, è insolito che venga richiesta l'SVD completa. Possiamo però utilizzare delle varianti della SVD che re richiedono un approssimazione a basso rango considerando il valore singolare più grande che contiene la maggior parte dell'informazione contenuta nell'immagine. I metodi di SVD ridotta sfruttano una versione ottimizzata della Decomposizione ai Valori Singolari al fine di ottenere una rappresentazione approssimata di una matrice originale, ma con una dimensione ridotta. Questa tecnica è particolarmente vantaggiosa quando si lavora con matrici di grandi dimensioni, poiché consente di ridurre la complessità computazionale. La forma della SVD ridotta è del tipo:

$$M \approx U_k \Sigma_k V_k^T$$

con:

- U_k è una matrice contenente solo le prime k colonne di U.
- Σ_k è una matrice diagonale contenente solo i primi k valori singolari.

- V_k^T è la trasposta di una matrice contenente solo le prime k colonne di V.

Queste varianti della SVD trovano applicazioni in diversi contesti tra cui la compressione delle immagini. Queste tecniche sono fondamentali in contesti in cui si desidera avere una buona precisione nell'analisi con la necessità di gestire matrici di grandi dimensioni in modo più efficiente. Abbiamo 3 varianti di SVD con approssimazione a basso rango:

8.1 SVD Sottile

L'SVD sottile di una matrice M è dato da

$$M = U_k \Sigma_k V_k^*, \tag{3}$$

dove $k = \min(m, n)$.
Le matrici U_k e V_k contengono solo le prime k colonne di U e V, e Σ_k contiene solo i primi k valori singolari da Σ. La matrice U_k è quindi $M \times k$, Σ_k è $k \times k$ diagonale, e V_k^* è $k \times N$.
L'SVD Sottile utilizza significativamente meno spazio e tempo di calcolo se $k \ll \max(m, n)$. La prima fase del suo calcolo sarà solitamente una fattorizzazione QR di M, che in questo caso può rendere il calcolo notevolmente più rapido. Implementiamolo in MatLab:

```
file = '000007.dcm';
image = dicomread(file);
% Calcoliamo la SVD sottile
[U, S, V] = svd(double(image), 'econ');
% Visualizziamo l'immagine originale
subplot(2, 3, 2);
imshow(image, []);
title('Immagine Originale');
% Visualizziamo l'immagine dopo aver applicato SVD sottile
immagine_ricostruita = U * S * V';
subplot(2, 3, 3);
imshow(immagine_ricostruita, []);
title('Immagine Ricostruita con SVD sottile');
```

Il risultato finale di questa implementazione in Figure 16 è che otteniamo due foto identiche

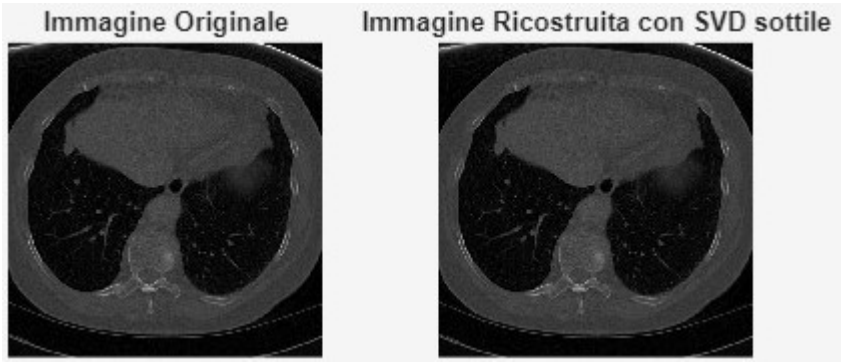

Figure 16: Implementazione con SVD Sottile

8.2 SVD Compatta

L'SVD compatta di una matrice M è dato da

$$M = U_R \Sigma_R V_R^*. \tag{4}$$

Solo gli R vettori colonna di U e R vettori riga di V^* corrispondenti ai valori singolari diversi da zero in Σ_R vengono calcolati. I vettori rimanenti di U e V^* non vengono calcolati. Questo è più rapido ed economico del sottile SVD se $R \ll \min(m, n)$. La matrice U_R è quindi $M \times R$, Σ_R è $R \times R$ diagonale, e V_R^* è $R \times N$.

```
file = '000007.dcm';
image = dicomread(file);
% Calcoliamo la SVD Compatta
[U, S, V] = svd(double(image));
% Specifichiamo il numero di valori singolari
r = 50;
Ur = U(:, 1:r);
Sr = S(1:r, 1:r);
Vr = V(:, 1:r);
% Visualizziamo l'immagine originale
subplot(2, 3, 2);
imshow(image, []);
title('Immagine Originale');
% Visualizziamo l'immagine dopo la SVD Compatta
immagine_ricostruita = Ur * Sr * Vr';
subplot(2, 3, 3);
imshow(immagine_ricostruita, []);
title(['Immagine Ricostruita con  SVD Compatta']);
```

Di questa implementazione, il risultato finale in Figure 17 consiste nel confronto visivo tra le due immagini, di cui quella ricostruita e' la migliore approssimazione dell'originale ottenuta utilizzando solo le 50 componenti principali della SVD.

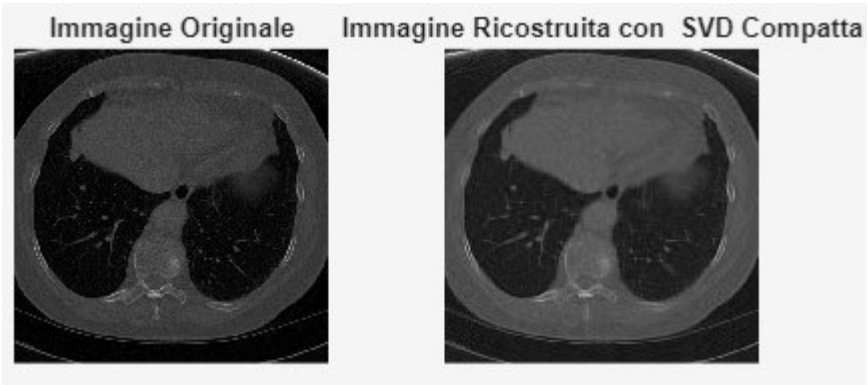

Figure 17: Implementazione SVD Compatta

8.3 SVD Troncata

In molte applicazioni il numero R dei valori singolari diversi da zero è grande, rendendo persino l'SVD Compatta poco pratico da calcolare. In tali casi, potrebbe essere necessario troncare i valori singolari più piccoli per calcolare solo $T \ll R$ valori singolari diversi da zero. La SVD troncata non è più una decomposizione esatta della matrice originale M, ma piuttosto fornisce l'approssimazione ottimale della matrice di basso rango $\widetilde{M}$ da qualsiasi matrice di rango fisso T:

$$\widetilde{M} = U_T \Sigma_T V_T^*, \tag{5}$$

dove matrice U_T è $M \times T$, Σ_T è $T \times T$ diagonale, e V_T^* è $T \times N$. Solo i T vettori colonna di U e T vettori riga di V^* corrispondente ai T valori singolari più grandi Σ_T vengono calcolati. Questo può essere molto più rapido ed economico rispetto al compatto SVD se $T \ll R$, ma richiede un set di strumenti di risoluzione numerica completamente diverso.

```
file = '000007.dcm';
image = dicomread(file);
% Calcoliamo l' SVD Troncata dell'immagine
[U, S, V] = svd(double(image));
% Numero di valori singolari da mantenere
t = 50;
Ut = U(:, 1:t);
St = S(1:t, 1:t);
Vt = V(:, 1:t);
% Calcoliamo l'approssimazione dell'immagine originale
troncamento = Ut * St * Vt';
% Visualizziamo l'immagine originale
subplot(2, 3, 2);
imshow(image, []);
title('Immagine Originale');
% Visualizziamo l'immagine dopo SVD Troncata
```

```
subplot(2, 3, 3);
imshow(troncamento, []);
title('Immagine Approssimata con SVD Troncata');
```

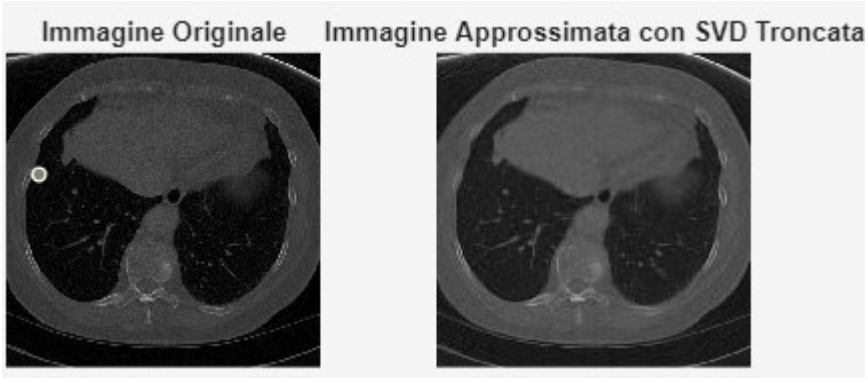

Figure 18: Implementazione SVD Troncata

Troncando la SVD ai primi t valori singolari (in questo caso $t = 50$), si ottenga un'immagine che rappresenta un compromesso tra la compressione dei dati (si memorizzano solo i fattori U_t, S_t, V_t) e la fedeltà visiva all'originale.

8.4 Illustrazione

Dopo aver implementato questi metodi analiziamone un illustrazione in Figure 19:

- **SVD Completa**: Decomposizione esatta senza riduzioni.
- **SVD Sottile**: Rimuove le colonne di U non corrispondenti alle righe di V^*. Ottimizzato quando le dimensioni m e n sono diverse.
- **SVD Compatta**: Rimuove i valori singolari evanescenti (uguali a zero) e le colonne/righe corrispondenti in U e V^*. Utile quando il rango della matrice è noto ed è $\ll \min(m, n)$.
- **SVD Troncata**: Mantiene solo i T valori singolari più grandi e le colonne/righe corrispondenti in U e V^*. Questa è l'approssimazione ottimale a basso rango ($T < rank(M)$), usata per la compressione.

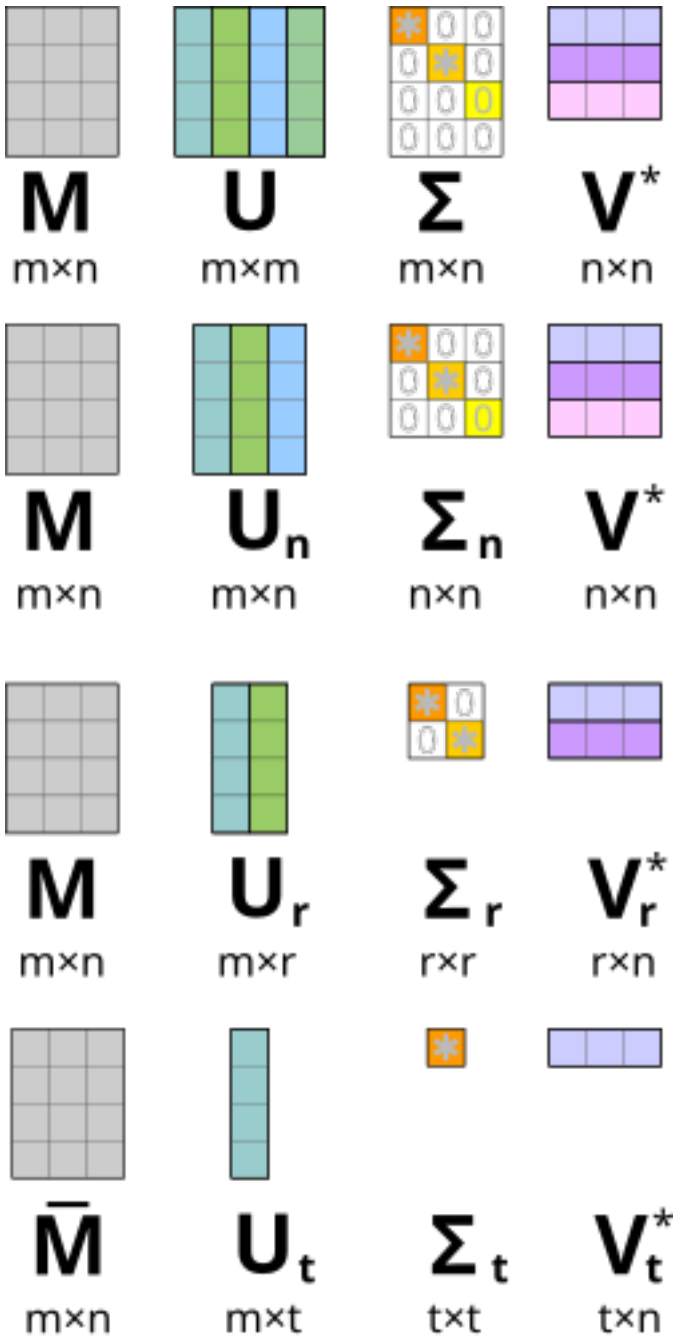

Figure 19: Confronto tra le SVD

9 Conclusioni

Il presente lavoro ha analizzato le potenzialità della Decomposizione ai Valori Singolari (SVD) come strumento versatile e robusto per l'elaborazione di immagini mediche. Attraverso l'implementazione in ambiente MatLab, è stato possibile dimostrare come la teoria matematica delle matrici trovi riscontri immediati e di grande valore clinico.

In sintesi, i risultati ottenuti confermano che:

- **Efficienza e Compressione:** L'approssimazione a basso rango permette di ridurre drasticamente l'occupazione di memoria senza compromettere le informazioni diagnostiche essenziali, facilitando lo storage e la trasmissione dei dati in contesti di telemedicina.

- **Qualità e Denoising:** La separazione tra lo spazio dei dati (segnale utile) e lo spazio del rumore consente un filtraggio selettivo che migliora la nitidezza delle strutture anatomiche, distinguendo efficacemente tra componenti *lisce* e componenti *rugose*.

- **Sicurezza e Integrità:** L'applicazione del watermarking basato su SVD si è dimostrata una soluzione ottimale per garantire l'autenticità del dato clinico, offrendo un'elevata robustezza contro le manipolazioni e una totale impercettibilità visiva.

In conclusione, nonostante l'elevato costo computazionale richiesto per matrici di grandi dimensioni, l'utilizzo delle varianti ottimizzate (Sottile, Compatta e Troncata) rende la SVD una metodologia imprescindibile capace di coniugare rigore matematico e necessità pratiche del settore sanitario.

10 Riferimenti

- Wikipedia (EN) - Singular Value Decomposition (SVD)
- Dataset per Volumetric DICOM
- COVID-19 Radiography Database
- SVD Based Image Processing Applications
- On the Power of Truncated SVD for General High-rank Matrix Estimation Problems
- A multidimensional image quality measure using Singular Value Decomposition
- SVD based Image Watermarking Scheme